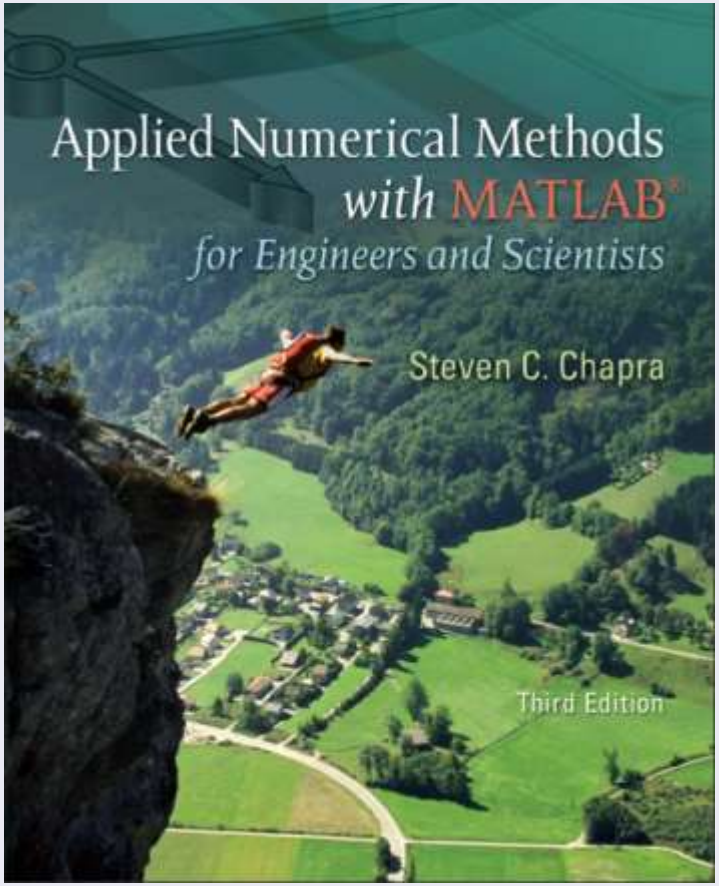


Part 3
Chapter 12

Iterative Methods

PowerPoints organized by Dr. Michael R. Gustafson II, Duke University

All images copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

Chapter Objectives

- Understanding the difference between the Gauss-Seidel and Jacobi methods.
- Knowing how to assess diagonal dominance and knowing what it means.
- Recognizing how relaxation can be used to improve convergence of iterative methods.
- Understanding how to solve systems of nonlinear equations with successive substitution and Newton-Raphson.

Gauss-Seidel Method

- The *Gauss-Seidel method* is the most commonly used iterative method for solving linear algebraic equations [*A*]{*x*}={*b*}.
- The method solves each equation in a system for a particular variable, and then uses that value in later equations to solve later variables. For a 3x3 system with nonzero elements along the diagonal, for example, the jth iteration values are found from the j-1th iteration using:

$$x_1^j = \frac{b_1 - a_{12}x_2^{j-1} - a_{13}x_3^{j-1}}{a_{11}}$$

$$x_2^j = \frac{b_2 - a_{21}x_1^{j} - a_{23}x_3^{j-1}}{a_{22}}$$

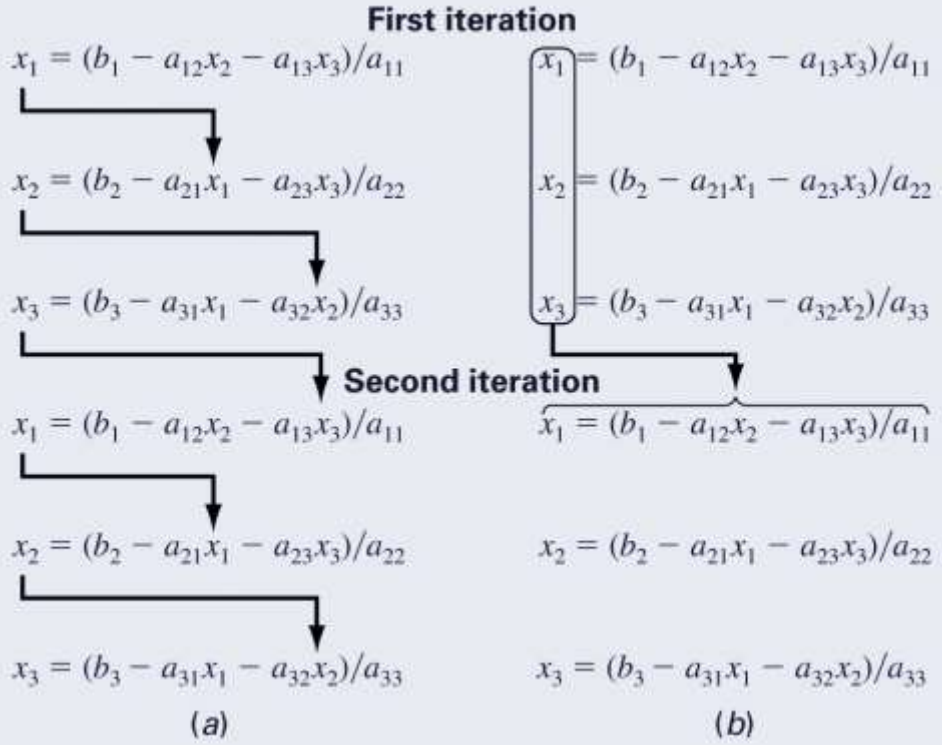
$$x_3^j = \frac{b_3 - a_{31}x_1^{j} - a_{32}x_2^{j}}{a_{33}}$$

Jacobi Iteration

- The *Jacobi iteration* is similar to the Gauss-Seidel method, except the j-1th information is used to update all variables in the jth iteration:

a) Gauss-Seidel

b) Jacobi



Convergence

- The convergence of an iterative method can be calculated by determining the relative percent change of each element in {*x*}. For example, for the *i*th element in the *j*th iteration,

$$\varepsilon_{a,i} = \left| \frac{x_i^j - x_i^{j-1}}{x_i^j} \right| \times 100\%$$

- The method is ended when all elements have converged to a set tolerance.

Diagonal Dominance

- The Gauss-Seidel method may diverge, but if the system is *diagonally dominant*, it will definitely converge.
- Diagonal dominance means:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}|$$

MATLAB Program

```
function x = GaussSeidel(A,b,es,maxit)
% GaussSeidel: Gauss Seidel method
%   x = GaussSeidel(A,b): Gauss Seidel without relaxation
% input:
%   A = coefficient matrix
%   b = right hand side vector
%   es = stop criterion (default = 0.00001%)
%   maxit = max iterations (default = 50)
% output:
%   x = solution vector

if nargin<2,error('at least 2 input arguments required'),end
if nargin<4|isempty(maxit),maxit=50;end
if nargin<3|isempty(es),es=0.00001;end
[m,n] = size(A);
if m~=n, error('Matrix A must be square'); end
C = A;
for i = 1:n
  C(i,i) = 0;
  x(i) = 0;
end
x = x';
for i = 1:n
  C(i,1:n) = C(i,1:n)/A(i,i);
end
for i = 1:n
  d(i) = b(i)/A(i,i);
end
iter = 0;
while (1)
  xold = x;
  for i = 1:n
    x(i) = d(i)-C(i,:)*x;
    if x(i) ~= 0
      ea(i) = abs((x(i) - xold(i))/x(i)) * 100;
    end
  end
  iter = iter+1;
  if max(ea)<=es | iter >= maxit, break, end
end
```

Relaxation

- To enhance convergence, an iterative program can introduce *relaxation* where the value at a particular iteration is made up of a combination of the old value and the newly calculated value:

$$x_i^{\text{new}} = \lambda x_i^{\text{new}} + (1-\lambda) x_i^{\text{old}}$$

  where λ is a weighting factor that is assigned a value between 0 and 2.
  - $0<\lambda<1$: underrelaxation
  - $\lambda=1$: no relaxation
  - $1<\lambda\leq 2$: overrelaxation

Nonlinear Systems

- Nonlinear systems can also be solved using the same strategy as the Gauss-Seidel method - solve each system for one of the unknowns and update each unknown using information from the previous iteration.
- This is called *successive substitution*.

Newton-Raphson

- Nonlinear systems may also be solved using the Newton-Raphson method for multiple variables.
- For a two-variable system, the Taylor series approximation and resulting Newton-Raphson equations are:

$$f_{1,i+1} = f_{1,i} + \left(x_{1,i+1} - x_{1,i}\right)\frac{\partial f_{1,i}}{\partial x_1} + \left(x_{2,i+1} - x_{2,i}\right)\frac{\partial f_{1,i}}{\partial x_2}$$

$$x_{1,i+1} = x_{1,i} - \frac{f_{1,i}\dfrac{\partial f_{2,i}}{\partial x_2} - f_{2,i}\dfrac{\partial f_{1,i}}{\partial x_2}}{\dfrac{\partial f_{1,i}}{\partial x_1}\dfrac{\partial f_{2,i}}{\partial x_2} - \dfrac{\partial f_{1,i}}{\partial x_2}\dfrac{\partial f_{2,i}}{\partial x_1}}$$

$$f_{2,i+1} = f_{2,i} + \left(x_{1,i+1} - x_{1,i}\right)\frac{\partial f_{2,i}}{\partial x_1} + \left(x_{2,i+1} - x_{2,i}\right)\frac{\partial f_{2,i}}{\partial x_2}$$

$$x_{2,i+1} = x_{2,i} - \frac{f_{2,i}\dfrac{\partial f_{1,i}}{\partial x_1} - f_{1,i}\dfrac{\partial f_{2,i}}{\partial x_1}}{\dfrac{\partial f_{1,i}}{\partial x_1}\dfrac{\partial f_{2,i}}{\partial x_2} - \dfrac{\partial f_{1,i}}{\partial x_2}\dfrac{\partial f_{2,i}}{\partial x_1}}$$

MATLAB Program

```
function [x,f,ea,iter]=newtmult(func,x0,es,maxit,varargin)
% newtmult: Newton-Raphson root zeroes nonlinear systems
%  [x,f,ea,iter]=newtmult(func,x0,es,maxit,p1,p2,...):
%    uses the Newton-Raphson method to find the roots of
%    a system of nonlinear equations
% input:
%   func = function handle to function that returns f and J
%   x0 = initial guess
%   es = desired percent relative error (default = 0.0001%)
%   maxit = maximum allowable iterations (default = 50)
%   p1,p2,... = additional parameters used by function
% output:
%   x = vector of roots
%   f = vector of functions evaluated at roots
%   ea = approximate percent relative error (%)
%   iter = number of iterations

if nargin<2,error('at least 2 input arguments required'),end
if nargin<3||isempty(es),es=0.0001;end
if nargin<4||isempty(maxit),maxit=50;end
iter = 0;
x=x0;
while (1)
  [J,f]=func(x,varargin{:});
  dx=J\f;
  x=x-dx;
  iter = iter + 1;
  ea=100*max(abs(dx./x));
  if iter>=maxit||ea<=es, break, end
end
```